VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
library work;
use work.p_constants.all;
```

ARCHITECTURE DECLARATIONS

```
constant c_tb_number_switches : integer := g_tb_number_switches;
constant c_tb_number_buttons : integer := g_tb_number_buttons;
type sw_t is array (0 to 0) of std_logic_vector (c_tb_number_switches - 1 downto 0);
type btn_t is array (0 to 0) of std_logic_vector (c_tb_number_buttons - 1 downto 0);
type segment_t is array (0 to 0) of std_logic_vector (6 downto 0);
```

clock

```
clock_generate_10ns : process is
begin
  ck <= '0'; wait for 5 ns;
  ck <= '1'; wait for 5 ns;
end process;
```

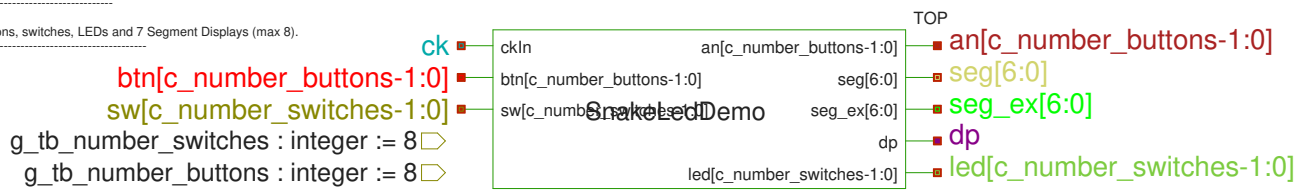
```
--------------------------------------------------------------------------------
--
--                  O o
--                    L
--------------------OO-------OO-----------------------------------------------
-- Originally header from VHD file
--------------------------------------------------------------------------------
----------------------------------------------------------------------------------
-- Company: Digilent
-- Engineer: Mircea Dabacan
--
-- Create Date: 12:36:33 12/06/05
-- Design Name: Nexys2 board demo
-- Module Name: Demo - Behavioral
-- Project Name:
-- Target Device: XC3S500E
-- Tool versions: Xilinx ISE 9.2
-- Description: This file contains the demo for the 7 Segment Display and
--                              and the leds on the Nexys 2 Board.
--
--Each Led is assigned to a switch on the board. When a switch is moved to
--the HIGH the coresponding led will turn on slowly. When it is moved to
--the LOW position the led will turn off slowly. This is done using PWM's.
--
--The Snake is the demo on the 7 Segment Display. A snake apears on one digit.
--The head of the snake is the represented by the brightest line and it's tale
--by the least bright line. When a button is pressed the snake moves to the
--coresponding digit.
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------

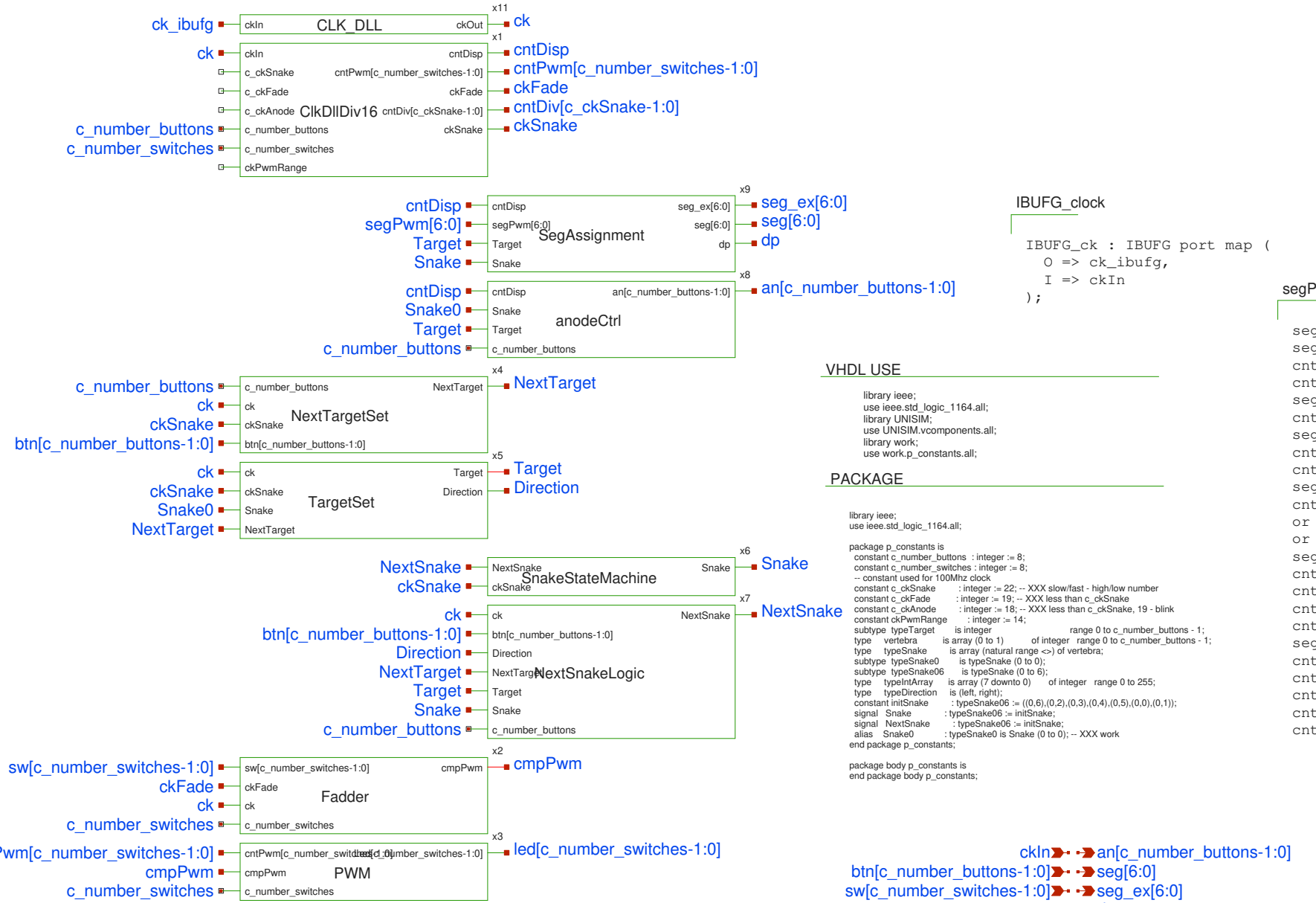
--  Small feature to use more buttons, switches, LEDs and 7 Segment Displays (max 8).
--------------------------------------------------------------------------------
```

stimuli

```
stimulus_sw : process is
begin
  sw <= (others => '0');
  wait for 20 ns;
  sw(1) <= '1';
  sw(5) <= '1';
  wait for 100 ms;
  sw(1) <= '0';
  sw(5) <= '0';
  wait for 100 ns;
  wait;
end process;

stimulus_btn : process is
  procedure push_btn (index : in integer) is
  begin
    btn (index) <= '1'; wait for 10 ns; btn (index) <= '0';
    wait for 1000 ms;
  end procedure;
begin
  btn <= (others => '0');
  push_btn (1);
  push_btn (5);
  push_btn (1);
  push_btn (3);
  push_btn (2);
  push_btn (4);
  push_btn (1);
  report "tb done" severity failure;
end process;
```





For synthesis project, COMMENT line "signal Snake0 : typeSnake0" (ARCHITECTURE DECLARATIONS) in VHDL netlist file.
(Tested on board Nexys 2 1200 E - part xc3s1200e-fg320-4)

ckAnode

```
p_ckAnode : process (ckAnode_i) is
begin
  if ckAnode_i'event and ckAnode_i = '1' then
    if (cntDisp_i = c_number_buttons - 1) then
      cntDisp_i <= 0;
    else
      cntDisp_i <= cntDisp_i + 1;
    end if;
  end if;
end process;
cntDisp <= cntDisp_i;
```

ARCHITECTURE DECLARATIONS

```
signal cntDiv_i : std_logic_vector(c_ckSnake - 1 downto 0) := (others => '0');
signal ckAnode_i : std_logic := '0';
signal cntDisp_i : typeTarget := 0;
```

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
library work;
use work.p_constants.all;
```

ckDivider

```
p_ckDivider : process (ckIn) is
begin
  if rising_edge (ckIn) then
    cntDiv_i <= cntDiv_i + '1';
  end if;
end process;
cntPwm <= cntDiv_i (7 + ckPwmRange downto ckPwmRange);
ckFade <= cntDiv_i (c_ckFade - 1);
ckSnake <= cntDiv_i (c_ckSnake - 1);
ckAnode_i <= cntDiv_i (c_ckAnode - 1);
cntDiv <= cntDiv_i; -- XXX for cntPwm in top
```

c_ckSnake : integer := 22
c_ckAnode : integer := 18
c_number_buttons : integer := 1
c_number_switches : integer := 1
ckPwmRange : integer := 14
c_ckFade : integer := 18

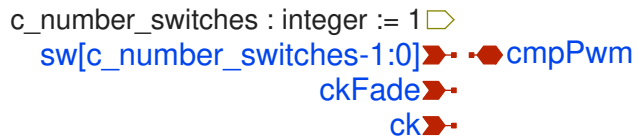
ckIn
- cntDiv[c_ckSnake-1:0]
- cntDisp
- cntPwm[c_number_switches-1:0]
- ckFade
- ckSnake

Fadder

```
p_Fadder : process (ckFade) is
begin
  for i in 0 to c_number_switches - 1 loop -- for each LED
    if ckFade'event and ckFade='1' then
      if sw(i) = '1' and cmpPwm(i) /= 255 then
        -- coresponding switch ON, not yet full intensity
        cmpPwm(i) <= cmpPwm(i)+1; -- increasing intensity
      elsif sw(i) = '0' and  cmpPwm(i) /= 0 then
        -- coresponding switch OFF, not yet full dark
        cmpPwm(i) <= cmpPwm(i)-1; -- decreasing intensity
      end if;
    end if;
  end loop;
end process;
```

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
library work;
use work.p_constants.all;
```



PWM

```
p_PWM : process (cntPwm, cmpPwm) is
begin
  for i in 0 to c_number_switches - 1 loop -- for each LED
    if conv_integer (cntPwm) < cmpPwm (i) then
      -- counter value less than reference
      Led(i) <= '1'; -- LED ON
    else
      -- counter value greater than reference
      Led(i) <= '0'; -- LED OFF
    end if;
  end loop;
end process;
```

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
library work;
use work.p_constants.all;
```

c_number_switches : integer := 1

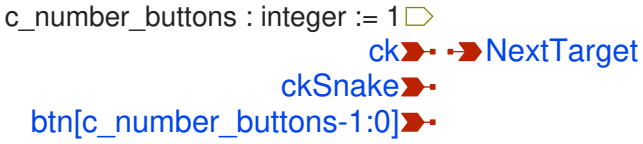
cntPwm[c_number_switches-1:0] → Led[c_number_switches-1:0]

cmpPwm

XSCHEM

2024-09-19 12:53:47

->netlisting xschemtest1/PWM.sch



VHDL USE

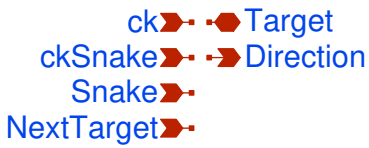
```
library ieee;
use ieee.std_logic_1164.all;
library work;
use work.p_constants.all;
```

NextTargetSet

```
--in case a button is pressed sets the next target for the snake
p_NextTargetSet : process (ckSnake) is
begin
  if ckSnake'event and ckSnake='1' then
    for i in 0 to c_number_buttons - 1 loop
      if btn(i) = '1' then
        NextTarget <= i;
      end if;
    end loop;
  end if;
end process;
```

TargetSet

```
p_TargetSet : process (ckSnake) is
begin
  if ckSnake'event and ckSnake='1' then
    if Snake(0) = (Target, 6) then -- already at target
      Target <= NextTarget;
      if NextTarget > Target then
        --sets the direction for the snake depending on
        --the position of the next target
        Direction <= left;
      elsif NextTarget < Target then
        Direction <= right;
      end if;
    end if;
  end if;
end process;
```



VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
library work;
use work.p_constants.all;
```

SnakeStateMachine

```
p_SnakeStateMachine : process (ckSnake)
begin
        if ckSnake'event and ckSnake='1' then
                Snake <= NextSnake;
        end if;
end process;
```

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
library work;
use work.p_constants.all;
```





NextSnakeLogic

```
p_NextSnakeLogic : process (
  Target, nextTarget, Direction,
  Snake(0), Snake(1), Snake(2), Snake(3), Snake(4), Snake(5), Snake(6)
) is
begin
  if Snake(0) = (Target, 6) then -- already at target
    if    NextTarget > Target then
      NextSnake <= ((Target,6),(Target,1),(Target,0),(Target,5),(Target,4),(Target,3),(Target,2)); -- spinning to left
    elsif NextTarget < Target then
      NextSnake <= ((Target,6),(Target,5),(Target,0),(Target,1),(Target,2),(Target,3),(Target,4)); -- spinning to left
    else
      NextSnake <= Snake; -- resting
    end if;
  else -- snake moving
    NextSnake (1 to 6) <= Snake (0 to 5);
    if Direction = left then -- snake moving left
      if        Snake (0)  = (Target - 1    , 6)   then
            nextSnake (0) <= (Target        , 2);
      elsif     Snake (0)  = (Target        , 2)   then -- roling in target digit
            nextSnake (0) <= (Target        , 3);
      elsif     Snake (0)  = (Target        , 3)   then -- roling in target digit
            nextSnake (0) <= (Target        , 4);
      elsif     Snake (0)  = (Target        , 4)   then -- roling in target digit
            nextSnake (0) <= (Target        , 5);
      elsif     Snake (0)  = (Target        , 5)   then -- roling in target digit
            nextSnake (0) <= (Target        , 0);
      elsif     Snake (0)  = (Target        , 0)   then -- roling in target digit
            nextSnake (0) <= (Target        , 1);
      elsif     Snake (0)  = (Target        , 1)   then -- roling in target digit
            nextSnake (0) <= (Target        , 6);
      else
            nextsnake (0) <= Snake(0);
      end if;
      for i in c_number_buttons downto 2 loop
        if      Snake (0)  = (Target - i    , 6) then -- running to left
            NextSnake (0) <= (Target - i + 1, 6);
        end if;
      end loop;
    else -- snake moving right
      if        Snake (0)  = (Target + 1    , 6) then -- next right to target
            NextSnake (0) <= (Target        , 4);
      elsif     Snake (0)  = (Target        , 4) then -- roling in target digit
            NextSnake (0) <= (Target        , 3);
      elsif     Snake (0)  = (Target        , 3) then -- roling in target digit
            NextSnake (0) <= (Target        , 2);
      elsif     Snake (0)  = (Target        , 2) then -- roling in target digit
            NextSnake (0) <= (Target        , 1);
      elsif     Snake (0)  = (Target        , 1) then -- roling in target digit
            NextSnake (0) <= (Target        , 0);
      elsif     Snake (0)  = (Target        , 0) then -- roling in target digit
            NextSnake (0) <= (Target        , 5);
      elsif     Snake (0)  = (Target        , 5) then -- roling in target digit
            NextSnake (0) <= (Target        , 6);
      else
            NextSnake (0) <= Snake (6);
      end if;
      for i in c_number_buttons downto 2 loop
        if      Snake (0)  = (Target + i    , 6) then -- running to left
            NextSnake (0) <= (Target + i - 1, 6);
        end if;
      end loop;
    end if;
  end if;
end process;
```

c_number_buttons : integer := 1

ck → NextSnake

btn[c_number_buttons-1:0]

Direction

NextTarget

Target

Snake

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
library work;
use work.p_constants.all;
```

c_number_buttons : integer := 1

cntDisp

Snake

Target

an[c_number_buttons-1:0]

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
library work;
use work.p_constants.all;
```

anodeCtrl

ARCHITECTURE DECLARATIONS

```
signal an_i : std_logic_vector (c_number_buttons - 1 downto 0) := (others => '1');
```

```
--Only one digit is active at a certain moment, this is done by activating one anodeat a time.
--The change of the active digits is done very fast and it makes you belive that all digits
--are lit at the same time
an <= an_i xor "00001111"; -- XXX for the maximator expander
p_anodeCtrl : process (cntDisp, Target, Snake(0))
        variable ones : std_logic_vector (c_number_buttons - 1 downto 0) := (others => '1');
begin
        an_i <= ones;
        for i in 0 to c_number_buttons - 1 loop
                if (Snake(0) = (Target, 6)) then
                        an_i (Target) <= '0';
                elsif (cntDisp = i) then
                        an_i (cntDisp) <= '0';
                end if;
        end loop;
end process;
```

segAssignment

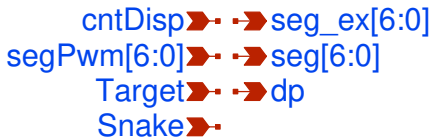
```
seg_ex <= seg_i xor "1111111"; -- XXX for the maximator expander
seg <= seg_i;
--The output segment vector is created here.
p_SegAssignment: process (
  cntDisp, segPwm, Target, Snake(0), Snake(1), Snake(2), Snake(3), Snake(4), Snake(5), Snake(6)
) is
begin
  seg_i <= "1111111";
  for k in 0 to 6 loop
    for i in 0 to 6 loop --loops all vertebras of the snake
      if Snake(i) = (cntDisp,k) then
        seg_i(k) <= segPwm(i);
      end if;
      if Target = cntDisp then -- current 7seg digit
        dp <= '0';  -- segment OFF
      else
        dp <= '1';      -- segment ON
      end if;
    end loop;
  end loop;
end process;
```

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
library work;
use work.p_constants.all;
```

ARCHITECTURE DECLARATIONS

```
signal seg_i : std_logic_vector (6 downto 0) := (others => '0');
```



DCM

```
CLK_DDL_BUFG_inst : BUFG port map (
  O => ckBuf2,
  I => ckBuf1
);

RST_SRL16_inst : SRL16
generic map (
  INIT => x"FFFF"
)
port map (
  D => '0',
  CLK => ckIn,
  A0 => '1',
  A1 => '1',
  A2 => '1',
  A3 => '1',
  Q => rst
);

CLK_DLL_inst : DCM port map (
  CLK0     => ckBuf1,
  CLKFB    => ckBuf2,
  CLKDV    => open,
  CLKIN    => ckIn,
  RST      => rst,
  CLK180   => open,
  CLK270   => open,
  CLK2X    => ckOut,
  CLK90    => open,
  LOCKED   => open
);
```

ARCHITECTURE DECLARATIONS

```
attribute CLKDV_DIVIDE : real;
attribute DUTY_CYCLE_CORRECTION : boolean;
attribute STARTUP_WAIT : boolean;
attribute SIM_MODE : string;
--attribute LOC : string;
signal ckBuf1,ckBuf2 : std_logic;

signal rst : std_logic;

component DCM
generic (
  CLKDV_DIVIDE : real := 16.0; -- (1.5, 2.0, 2.5, 3.0, 4.0, 5.0, 8.0, 16.0)
  DUTY_CYCLE_CORRECTION : Boolean := TRUE;
  STARTUP_WAIT : boolean := FALSE;
  SIM_MODE : string := "FAST"); -- XXX must use reset
  --STARTUP_WAIT : boolean := FALSE;
  --LOC : string := "DCM_X2Y0");
port (
  CLK0 : out STD_LOGIC;
  CLK180 : out STD_LOGIC;
  CLK270 : out STD_LOGIC;
  CLK2X : out STD_LOGIC;
  CLK90 : out STD_LOGIC;
  CLKDV : out STD_LOGIC;
  LOCKED : out STD_LOGIC;
  CLKFB : in STD_LOGIC;
  CLKIN : in STD_LOGIC;
  RST : in STD_LOGIC
);
end component;
```

VHDL USE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
library UNISIM;
use UNISIM.vcomponents.all;
```

ckIn

ckOut